Capítulo 1 — Fundamentos del desarrollo web y diseño de aplicaciones

Capítulo 1. Fundamentos del desarrollo web y diseño de aplicaciones

> Unidad Didáctica: **Lenguajes para el Desarrollo y Soporte de Sistemas** Tema general: **Diseño de Aplicaciones** Referencia base: Semana 1 — PPT_Semana_1.pptx

---

1.1. Presentación del capítulo

Este primer capítulo introduce al estudiante al ecosistema del desarrollo web moderno y sienta las bases metodológicas, conceptuales y operativas sobre las que se construirán los capítulos posteriores del libro. El enfoque es doble: por un lado, familiarizar al lector con las herramientas y flujos de trabajo que utilizan hoy las y los desarrolladores profesionales (control de versiones, entornos colaborativos, editores modernos, despliegue en la nube); por otro, presentar el proceso de experiencia de usuario (UX) que antecede a toda implementación técnica seria.

Al finalizar este capítulo, el lector será capaz de:

1. Comprender el alcance y los productos de aprendizaje de la unidad didáctica.
2. Comunicarse y coordinar en entornos virtuales de manera asincrónica.
3. Utilizar Git y GitHub como sistema de control de versiones de un proyecto web real.
4. Configurar una herramienta de desarrollo de software moderna (Visual Studio Code) y publicar una primera página en GitHub Pages.
5. Reconocer las etapas del proceso UX aplicado al diseño de páginas y aplicaciones web.

1.2. Indicadores de logro de la unidad didáctica

La unidad se articula en torno a cuatro indicadores de logro (IL) que guiarán el trabajo de las semanas siguientes. El capítulo 1 es la puerta de entrada a todos ellos.

Código	Indicador de logro
IL1	Aplica elementos, etiquetas y atributos de HTML para elaborar páginas web utilizando la herramienta de desarrollo de software Visual Studio Code, de manera colaborativa como parte de un equipo de trabajo.
IL2	Aplica estilos CSS a páginas web de acuerdo a un diseño gráfico definido previamente, de manera colaborativa como parte de un equipo de trabajo.
IL3	Utiliza el lenguaje de programación JavaScript para elaborar páginas web interactivas, respetando su sintaxis y participando de manera colaborativa como parte de un equipo de trabajo.
IL4	Despliega un proyecto web en el entorno de Cloud Computing Firebase, de manera colaborativa como parte de un equipo de trabajo.

Los cuatro indicadores comparten dos rasgos transversales: la **colaboración en equipo** y el **respeto por un flujo de trabajo profesional**. Ambos se cultivan desde este primer capítulo.

1.3. Metodología: Aprendizaje Basado en Proyectos

La unidad adopta la metodología de **Aprendizaje Basado en Proyectos (ABP)**. En lugar de aprender cada tecnología de forma aislada, el estudiante integra los conocimientos en un único proyecto que acompaña toda la unidad.

1.3.1. El proyecto guía: Diseño de una Tienda Virtual

El proyecto busca que, con los recursos que ofrecen los lenguajes HTML, CSS y JavaScript, los estudiantes desarrollen una plataforma comercial que dé solución a un negocio real o simulado. Para ello se propone implementar una **Tienda Virtual** orientada a pequeñas empresas y microempresas, que ofrezca una solución flexible, escalable y de alta disponibilidad, utilizando tecnologías disruptivas y soluciones basadas en la nube.

La elección de una tienda virtual no es arbitraria: obliga al equipo a enfrentarse a problemas reales de maquetado, navegación, catálogos, formularios y despliegue, que cubren de forma natural los cuatro indicadores de logro.

1.3.2. Modo de trabajo

Durante toda la unidad el modo de trabajo es **grupal**: los estudiantes conforman un único equipo por proyecto y lo mantienen hasta la presentación final. Cada integrante asume responsabilidades, pero todo el código del proyecto se consolida en un mismo repositorio compartido.

1.4. Contenidos de la semana 1

Esta semana trabajaremos cuatro bloques que, en conjunto, construyen la base operativa del curso:

1. **Comunicación en entornos virtuales.**
2. **Uso del sistema de control de versiones GitHub.**
3. **Uso de la herramienta de desarrollo de software.**
4. **Diseño de páginas web (proceso UX).**

   Logro de la sesión. Al finalizar la sesión, el estudiante utiliza la herramienta GitHub de manera eficiente.

1.5. Comunicación en entornos virtuales

El desarrollo de software profesional rara vez ocurre en un aula sincrónica: lo habitual es trabajar de forma **asincrónica**, con personas ubicadas en distintos horarios y geografías. Por eso, antes de escribir una sola línea de código, los estudiantes deben aprender a coordinarse como lo hace un equipo distribuido.

En esta unidad los estudiantes realizarán de forma asincrónica las siguientes actividades:

- Acordar canales oficiales de comunicación del equipo (chat, correo, videollamadas).
- Programar y compartir reuniones de avance del proyecto.
- Documentar decisiones tomadas, acuerdos y pendientes.

Estas herramientas permiten que los integrantes del grupo coordinen avances del proyecto sin depender de encuentros presenciales obligatorios.

1.6. Sistema de control de versiones con GitHub

El **sistema de control de versiones** es la columna vertebral del trabajo colaborativo en software. En este curso se utilizará **Git** como sistema local y **GitHub** como plataforma remota. Los objetivos de esta sección son que cada estudiante sea capaz de:

- Crear una cuenta en GitHub o acceder a una existente.
- Crear un repositorio en GitHub.
- Instalar GitHub Desktop en su máquina local.
- Clonar un repositorio de GitHub a un directorio local utilizando una herramienta de desarrollo de software.
- Crear una página `index.html` usando esa herramienta y publicarla en GitHub Pages.

1.6.1. Del repositorio individual al repositorio de proyecto

El camino recomendado es el siguiente:

1. Cada estudiante experimenta primero con su **propio repositorio** individual, donde practica los comandos y el flujo de publicación.
2. A continuación, el **coordinador del grupo** crea el repositorio del proyecto y lo comparte con sus compañeros de equipo.
3. Cada integrante **clona** ese repositorio grupal a su máquina para empezar a contribuir.

1.6.2. Flujo mínimo de comandos

Aunque la unidad recomienda apoyarse en GitHub Desktop, conviene conocer el flujo equivalente en línea de comandos, porque es el que se encuentra en cualquier entorno profesional:

```
# 1. Clonar el repositorio a la máquina local
git clone https://github.com/usuario/mi-proyecto.git
cd mi-proyecto

# 2. Crear una rama de trabajo
git checkout -b feature/pagina-inicio

# 3. Registrar los cambios
git add index.html
git commit -m "feat: primera version de la pagina de inicio"

# 4. Publicar la rama en GitHub
git push origin feature/pagina-inicio
```

Este flujo —clonar, ramificar, commitear, empujar— se repite cientos de veces a lo largo de un proyecto real y debe volverse natural para el estudiante.

1.6.3. Publicación con GitHub Pages

Una vez que el repositorio contiene un `index.html`, GitHub Pages permite publicarlo como sitio web estático sin instalar ningún servidor adicional. Esta primera publicación es importante porque:

- demuestra que el flujo completo funciona de extremo a extremo;
- proporciona una URL pública que los compañeros de equipo y el docente pueden revisar;
- introduce el concepto de despliegue continuo, que se profundizará más adelante en Firebase.

1.7. Herramienta de desarrollo de software

La unidad trabaja con **Visual Studio Code (VS Code)** como herramienta de desarrollo. El estudiante debe:

- Instalar VS Code en la computadora donde trabajará el proyecto antes de que finalice la semana.
- Familiarizarse con la apertura de carpetas (el repositorio clonado), la edición de archivos HTML y la integración con Git.

Existe una relación directa entre GitHub y VS Code: el editor permite visualizar los cambios pendientes, hacer commits, cambiar de rama y sincronizar con el repositorio remoto sin salir de la ventana de trabajo. Esta integración es la que convierte a Git de una herramienta “de consola” en un recurso cotidiano.

1.8. Avance de la evidencia de aprendizaje AA1

Durante esta semana, además de los conceptos, cada equipo debe producir el primer avance de su **evidencia AA1**:

- Los estudiantes se organizan en grupos para desarrollar su proyecto.
- El modo de trabajo es **grupal** y se mantiene durante todo el desarrollo.
- Se deben revisar los documentos de Descripción del proyecto y Lineamientos de evidencia AA1.
- Al finalizar la sesión, cada grupo debe tener creada una cuenta en GitHub con el repositorio del proyecto.
- Antes del cierre de la semana, todos los integrantes deben tener instalado Visual Studio Code en la máquina donde trabajarán.

1.9. Diseño de aplicaciones: el proceso UX

Tener control de versiones y un editor no es suficiente para construir una buena aplicación. Después de entrevistar al cliente y conocer sus requerimientos, es necesario **diseñar** cómo se verá y se comportará la aplicación cuando esté terminada.

El **diseño de aplicaciones** se entiende aquí como parte de un proceso más amplio llamado **UX (User eXperience — Experiencia de Usuario)**. Dentro de su fase de diseño se distinguen seis etapas:

1. Concepto, boceto y flujos.
2. Wireframes y prototipos.
3. Diseño visual e interacciones.
4. Documentación.
5. Desarrollo.

Las siguientes secciones desarrollan cada etapa.

1.9.1. Concepto, boceto y flujos

Antes de dibujar pantallas, hay que asegurarse de contar con información suficiente sobre lo que el cliente necesita. Con esa base, se elaboran **bocetos**, normalmente a mano, que fijan el concepto de la aplicación y representan de manera visual y rápida cómo se vería y funcionaría el producto cuando ya estuviera construido.

Los **flujos** acompañan a los bocetos: describen qué pantallas existen y cómo se enlazan entre sí cuando el usuario interactúa.

1.9.2. Wireframes

A partir de los bocetos se construyen **wireframes**: guías visuales —un conjunto de pantallas y flujos de navegación— que explican la estructura, la jerarquía y los elementos de un producto digital (típicamente una aplicación web o móvil). Un wireframe no pretende ser bonito; pretende ser claro.

Los flujos de navegación incorporados al wireframe definen qué sucede con las pantallas cuando el usuario interactúa con ellas (pulsar un botón, enviar un formulario, navegar a otra sección).

Herramientas habituales para wireframes: **SketchApp**, **Balsamiq** o **Adobe Illustrator**.

1.9.3. Prototipos

Los **prototipos** son la simulación de la aplicación: se construyen haciendo interactivas las pantallas diseñadas en el wireframe. Con un prototipo, los usuarios pueden navegar entre pantallas como si la aplicación ya estuviera construida, lo que permite validar decisiones de diseño antes de escribir código.

1.9.4. Diseño visual

Una vez validada la estructura, se pasa al **diseño visual**: colores, tipografías, íconos, imágenes y demás materiales gráficos de la aplicación —tanto si será de escritorio, web o móvil—.

Es útil distinguir dos roles:

- **UX Designer (Diseñador de Experiencia de Usuario):** participa en todo el proceso del producto.
- **UI Designer (Diseñador de Interfaces de Usuario):** crea la parte visible del producto, diseña cada pantalla y sus elementos, y define la guía de estilos que se aplicará en la aplicación.

Herramientas habituales: **Photoshop**, **Sketch**, **Illustrator**, **Fireworks** y **Figma**.

1.9.5. Interacciones

Además de “cómo se ve”, una aplicación debe definir “cómo se usa”. El **UI Interaction Designer** se encarga de las animaciones y microinteracciones: qué hace una interfaz cuando el usuario realiza una acción sobre ella (tocar, deslizar, sacudir, arrastrar, etc.).

Herramientas habituales: **Adobe After Effects**, **Core Composer**, **Principle**, **Framer JS** y **Flinto**.

1.9.6. Documentación

En esta etapa se documentan todas las **guías de estilo** para que los equipos de desarrollo implementen correctamente el diseño visual —colores, imágenes, tipografías y demás recursos— y se describen también las interacciones del usuario con la aplicación. La documentación ayuda, además, a decidir qué **framework frontend** utilizar.

1.9.7. Desarrollo

Finalmente, después de investigar las necesidades del usuario, elaborar y documentar los flujos, definir el diseño visual y las interacciones, se construye la aplicación. Durante el desarrollo, los diseñadores del producto (product designers) participan activamente resolviendo dudas del equipo técnico.

Para el **desarrollo frontend** se utilizan los tres lenguajes que estructuran el resto del libro:

- **HTML**, para construir la estructura de las pantallas.
- **CSS**, para aplicarles estilos.
- **JavaScript**, para programar comportamiento e interacciones.

1.10. Actividades del capítulo

Actividad aplicativa 1 — Control de versiones en GitHub

Cada estudiante debe realizar, individualmente, el siguiente recorrido guiado por los materiales de la sesión:

1. Instalar Git — material: Sesión 1, Guía 4.
2. Crear una cuenta en GitHub — material: Sesión 1, Guía 6.
3. Crear un repositorio en GitHub — material: Sesión 1, Guía 7.
4. Clonar el repositorio en GitHub — material: Sesión 1, Guía 8.
5. Publicar en GitHub Pages — material: Sesión 1, Guía 9.

El docente aclara ideas, despeja dudas y explica la relación entre GitHub y Visual Studio Code.

Actividad aplicativa 2 — Bosquejo inicial del proyecto

El docente explica los conceptos básicos de diseño web siguiendo el material de la PPT — Semana 1. A continuación, cada grupo se plantea:

> ¿Cómo nos gustaría que sea el bosquejo de nuestras páginas web?

Y realiza un **bosquejo básico a modo de wireframe** de las páginas del proyecto.

Actividad grupal — Repositorio del proyecto

- El coordinador de cada grupo **crea el repositorio** del proyecto en GitHub y lo comparte con sus compañeros.
- Cada integrante **clona** el repositorio del proyecto a su máquina local.

1.11. Ideas principales

Para cerrar el capítulo, conviene tener presentes las siguientes ideas:

- **HTML** es el lenguaje básico para crear páginas web.
- Aprender HTML brinda acceso al mundo de la creación de contenido web.
- HTML es la base sobre la que se construyen otros lenguajes y tecnologías web, como **CSS** (estilos) y **JavaScript** (interactividad).
- HTML es un lenguaje relativamente simple y fácil de aprender.
- Aprender HTML puede abrir muchas oportunidades laborales en la industria tecnológica.
- HTML brinda la capacidad de crear y expresarse en línea.

1.12. Reflexión final

Cada grupo de proyecto debe responder, a modo de cierre del capítulo, las siguientes preguntas:

1. ¿Cuáles son las ventajas del uso de GitHub?
2. ¿De qué manera voy a incorporar lo aprendido en el avance de nuestro proyecto?

Las respuestas se registran en un enlace de plataforma colaborativa o, alternativamente, en hojas de papel, post-it u otro soporte físico, y se comparten con el resto de la clase.

1.13. Referencias

- Canziba, E. (2018). Hands-On UX Design for Developers. Birmingham, UK: Packt Publishing.

- GitHub. (s/f). Hoja de referencia para GitHub. https://training.github.com/downloads/es_ES/github-git-cheat-sheet.pdf
- Gómez, S. (s/f). Introducción a Git y GitHub. Universidad de Córdoba. https://www.uco.es/aulasoftwarelibre/wp-content/uploads/2015/11/git-cosfera-dia-1.pdf